


# COLLEGE PC MONITORING AND MANAGEMENT SYSTEM

**M. Noorul Aatif Rahman – 24619221**

**P. S. Jeevanandham – 24619242**

**P. Hariharan – 24619241**

## MINI PROJECT REPORT

Submitted to the

**SCHOOL OF ENGINEERING**

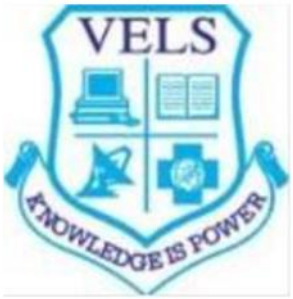
**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

in Partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY CSE**

**IN**

**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**




**VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)**

**CHENNAI – 600 117**

**OCTOBER 2026**

## BONAFIDE CERTIFICATE

Certified that this project report "**COLLEGE PC MONITORING AND MANAGEMENT SYSTEM**" is the bonafide work of **M. Noorul Aatif Rahman – 24619221, P. S. Jeevanandham – 24619242** and **P. Hariharan – 24619241**, who carried out the project under my supervision of the Department of Artificial Intelligence and Machine Learning, School of Engineering, Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

| SIGNATURE OF THE HOD | SIGNATURE OF THE SUPERVISOR |
|---|---|
| **Dr. K. Ulaga Priya M.E., Ph.D** | **Mrs. Divya Bairavi S ME., (PhD)** |
| Associate Professor & Head | Assistant Professor |
| Department of AI&ML | Department of AI&ML |
| School of Engineering | School of Engineering |
| VISTAS | VISTAS |
| Chennai | Chennai |

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

| INTERNAL EXAMINER | EXTERNAL EXAMINER |
|---|---|

## ACKNOWLEDGEMENT

It is a great pleasure for us to thank everyone who helped us to complete this mini project successfully. We thank our Founder and Chairman, Dr. Ishari K. Ganesh, for giving us a good learning place and the lab facilities that supported our work.

We are very thankful to our Head of the Department, **Dr. K. Ulaga Priya M.E.**,(PhD) Associate Professor and Head, Department of Artificial Intelligence and Machine Learning, for her steady support and kind guidance from the first idea of the project to its end.

We sincerely thank our project supervisor, **Mrs. Divya Bairavi S**, M.E., (Ph.D.), Assistant Professor, Department of AI&ML, for her patient guidance, useful advice and regular review of our work. Her suggestions helped us to improve the project and this report.

We also thank all the staff members of the department, the librarian and the staff of the computer centre for their help. We thank the authors of the research papers and the makers of the tools and libraries that we used. Finally, we thank our parents and friends for their love and support, without which we could not have finished this work.

**M. Noorul Aatif Rahman – 24619221**

**P. S. Jeevanandham – 24619242**

**P. Hariharan – 24619241**

# TABLE OF CONTENTS


| CHAPTER NO. | TABLE OF CONTENTS | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | **v** |
| | **LIST OF FIGURES** | **vi** |
| | **LIST OF TABLES** | **vii** |
| | **ABBREVIATIONS** | **viii** |
| **1** | **INTRODUCTION** | **1** |
| | 1.1 Overview of the Project | 1 |
| | 1.2 Problem Statement | 1 |
| | 1.3 Objectives | 1 |
| | 1.4 Scope | 2 |
| | 1.5 Report Organization | 2 |
| **2** | **LITERATURE REVIEW** | **3** |
| | 2.1 Background | 3 |
| | 2.2 Related Work | 3 |
| | 2.3 Base Paper and Conference Presentation | 4 |
| | 2.4 Research Gap and Project Position | 4 |
| | 2.5 Summary | 4 |
| **3** | **SYSTEM DESIGN** | **5** |
| | 3.1 Existing System | 5 |
| | 3.2 Proposed System | 5 |
| | 3.3 System Architecture | 5 |
| | 3.4 Architecture Flow | 6 |
| | 3.5 System Modules | 7 |
| | 3.6 User Roles | 8 |
| | 3.7 Functional Requirements | 9 |
| | 3.8 Non-Functional Requirements | 9 |
| | 3.9 Hardware and Software Requirements | 9 |
| **4** | **IMPLEMENTATION** | **11** |
| | 4.1 Development Approach | 11 |
| | 4.2 Administrator Desktop Application | 11 |
| | 4.3 Viewer Desktop Application and Monitoring Agent | 12 |
| | 4.4 Standalone PC Agent | 12 |
| | 4.5 Supabase Functions and Database | 13 |
| | 4.6 Security Design | 14 |
| | 4.7 Main Modules | 14 |

| CHAPTER NO. | TABLE OF CONTENTS | PAGE NO. |
|---|---|---|
| | 4.8 Data and Event Flow | 15 |
| | 4.9 Repository Structure | 15 |
| | 4.10 Build and Release | 16 |
| **5** | **TESTING, RESULTS AND DISCUSSION** | **17** |
| | 5.1 Testing Strategy | 17 |
| | 5.2 Results | 17 |
| | 5.2.1 Implemented Output | 17 |
| | 5.2.2 Default Monitoring Settings | 18 |
| | 5.2.3 What the Evidence Shows | 18 |
| | 5.3 Main Source Code | 19 |
| | 5.3.1 Shared Configuration (packages/config/src/index.ts) | 19 |
| | 5.3.2 System Monitor (agent/system-monitor.ts) | 19 |
| | 5.3.3 Agent Controller: Heartbeat, Commands and Activity (agent/controller.ts) | 20 |
| | 5.3.4 Command Handler (agent/command-handler.ts) | 21 |
| | 5.3.5 Offline Queue (agent/offline-queue.ts) | 22 |
| | 5.3.6 Screenshot Capture (agent/screenshot.ts) | 23 |
| | 5.3.7 Heartbeat Edge Function (supabase/functions/agent-heartbeat/index.ts) | 24 |
| | 5.3.8 Admin Command Service (admin-desktop/src/services/commands.ts) | 25 |
| | 5.3.9 Database Table and Security Rules (supabase/migrations) | 26 |
| | 5.4 Limitations | 27 |
| | 5.5 Discussion | 27 |
| **6** | **CONCLUSION AND FUTURE WORK** | **28** |
| | 6.1 Conclusion | 28 |
| | 6.2 Future Enhancement | 28 |
| | **REFERENCES** | **29** |
| | **CONFERENCE PAPER CERTIFICATES** | **30** |

# ABSTRACT

College computer labs have many computers. These computers must stay ready, safe and correctly set up for classes, exams and research work. Checking each computer by hand takes a lot of time and gives only a late picture of what is happening. This project builds a system that lets an administrator watch and manage all lab computers from one place.

The system uses two Electron desktop applications. The screens are made with React and TypeScript. Supabase is used for login, database storage, live updates and secure server functions. A viewer application is installed on each lab computer. It joins the system with a one-time code made by the administrator. After that, it sends the CPU and memory use every 15 seconds, notes which application is in use, takes a screenshot at a fixed gap (60 seconds by default), and carries out remote commands such as lock, logout, restart and shutdown.

The administrator application shows computers, labs, activity, sessions, applications, audit records and reports. Access is controlled in two places: in the screens and in the database through row-level security. This report explains the design, the main code and the testing plan of the system. It also states clearly which results are measured and which are only planned, so that no claim is made without proof.

**Keywords:** workstation monitoring, distributed systems, Electron, TypeScript, Supabase, endpoint telemetry, role-based access control, remote administration.

# LIST OF FIGURES


| FIGURE NO. | TITLE | PAGE NO. |
|---|---|---|
| Figure 1 | High-level system architecture | 6 |
| Figure 2 | Detailed process flow of the system | 7 |
| Figure 3 | Modules of the system and their functions | 8 |
| Figure 4 | Remote command workflow between admin, Supabase and the agent | 15 |
| Figure 5 | Repository application structure | 15 |
| Figure 6 | Sign-in screen of the College PC Admin application | 18 |

# LIST OF TABLES


| TABLE NO. | TITLE | PAGE NO. |
|---|---|---|
| Table 1 | Comparison of related monitoring systems | 3 |
| Table 2 | Main software components | 6 |
| Table 3 | Functional modules and their responsibilities | 8 |
| Table 4 | Hardware and software requirements | 9 |
| Table 5 | Project schedule (July to October 2026) | 11 |
| Table 6 | Edge Functions of the system | 13 |
| Table 7 | Main database tables | 13 |
| Table 8 | Main modules and their code locations | 14 |
| Table 9 | Test plan for the system | 17 |
| Table 10 | Default monitoring settings from the configuration package | 18 |

# ABBREVIATIONS

| ABBREVIATION | FULL FORM |
|---|---|
| **AI** | Artificial Intelligence |
| **API** | Application Programming Interface |
| **CPU** | Central Processing Unit |
| **CSV** | Comma-Separated Values |
| **DB** | Database |
| **HTTP** | Hypertext Transfer Protocol |
| **HTTPS** | Hypertext Transfer Protocol Secure |
| **IP** | Internet Protocol |
| **JSON** | JavaScript Object Notation |
| **JWT** | JSON Web Token |
| **ML** | Machine Learning |
| **OS** | Operating System |
| **PC** | Personal Computer |
| **RAM** | Random Access Memory |
| **RBAC** | Role-Based Access Control |
| **RLS** | Row-Level Security |
| **SQL** | Structured Query Language |
| **TLS** | Transport Layer Security |
| **UI** | User Interface |
| **URL** | Uniform Resource Locator |

# CHAPTER 1. INTRODUCTION

## 1.1 Overview of the Project

A computer lab is a shared place used for classes, practical work, online exams and research. When a lab has many computers, the lab in-charge must know a few simple things all the time. Is each computer switched on? How busy are its processor and memory? Which programs are open? Who is logged in? Did anything unusual happen?

This project builds a desktop-based system to answer these questions from one screen. It has two Electron applications [10]. The first is the **Admin application**, used by the administrator. The second is the **Viewer application**, which is installed on every lab computer that must be watched. Both applications talk directly to Supabase. There is no separate NestJS or Express server in this project.

Supabase [11] gives the project five things: user login, a PostgreSQL database with row-level security, live data updates, private storage for screenshots, and Edge Functions. Edge Functions are small server programs that do the jobs a desktop application should never do by itself, such as checking a computer's secret key.

## 1.2 Problem Statement

Checking lab computers by walking from one machine to the next is slow, and it shows only what is happening at that moment. A problem that starts after the visit stays unseen until someone complains. Many labs also use separate tools to check performance, user activity and reports, so it is hard to put the facts about one computer together.

The problem solved by this project is: **how can an administrator see the health, activity and sessions of many lab computers in one place, and safely control them from a distance?**

## 1.3 Objectives

The main objectives of the project are:

- To show all registered lab computers and their present status on one dashboard.
- To collect the CPU use and memory use of each computer at regular times.
- To record which application is in use and when a user session starts or ends.
- To allow only signed-in users to enter the system and to give each user only the rights of their role.

- To let the administrator lock, log out, restart or shut down a computer from a distance.
- To keep audit records, activity records and reports for later review.
- To use live updates and an offline queue so that the system reacts quickly and does not lose events when the network is down.

## 1.4 Scope

The system is made for college computers that are managed by the institution. Its main parts are the two Electron applications. It needs a Supabase project that has the database tables, security rules, storage bucket and Edge Functions already set up.

This report describes what is built in the project code. It does not claim that the system was tested on a large production lab, because no large-scale measurements were taken. Where a result is only a plan or a design goal, the report says so.

## 1.5 Report Organization

- **Chapter 1** gives the introduction, problem, objectives and scope.
- **Chapter 2** reviews related monitoring systems and the base paper.
- **Chapter 3** explains the system design, modules, roles and requirements.
- **Chapter 4** explains how the system is built, with its data and event flow.
- **Chapter 5** gives the testing plan, the results, the main source code, the limits and a discussion.
- **Chapter 6** gives the conclusion and future work.
- The **References** and the **Conference Paper Certificates** come at the end.

# CHAPTER 2. LITERATURE REVIEW

## 2.1 Background

A monitoring system collects facts from many computers and shows them together in one place. Most well-known tools were made for servers and clusters. They mainly watch numbers such as CPU load and memory. A college lab is different. Many students use the same computer at different times, so the administrator also wants to know who logged in, which programs were used and what actions were taken on the machine [7].

Because of this, a good lab system needs three things: simple health data, a record of user activity, and a safe way to control the machines from a distance.

## 2.2 Related Work

The project team studied several earlier systems. Table 1 lists them, shows what each one does, where it falls short for a college lab, and how this project tries to cover that gap.

**Table 1: Comparison of related monitoring systems**

| Author and title | Method used | Limitation | How this project helps |
|---|---|---|---|
| Al-Shaer, Abdel-Wahab and Maly [1] – HiFi: A New Monitoring Architecture for Distributed Systems Management | Agents spread in levels, with event filtering and event matching. | Too complex for one small lab. | A flat design: each PC talks to one central service. |
| McGilvary, Barker and Atkinson [2] – Ad Hoc Cloud Computing | A central server collects host state and shares work among volunteer hosts. | Hosts may not always be available. | Dedicated lab PCs with a heartbeat to show if each one is online. |
| Massie, Chun and Culler [3] – The Ganglia Distributed Monitoring System | Cluster monitoring with grouped results and a web page. | Made for clusters. No user session tracking or remote control. | Adds session and application records and remote commands. |
| Sottile and Minnich [4] – Supermon: A High-Speed Cluster Monitoring System | Nodes send metrics in a compact format. | Only metrics. Weak access control. | Login, roles and database security rules. |
| Zanikolas and Sakellariou [5] – A Taxonomy of Grid Monitoring Systems | A survey that groups grid monitoring systems by design. | A survey only. No lab implementation. | A working, lab-focused system built on the surveyed ideas. |
| Barth [6] – Nagios: System and Network Monitoring | Plug-ins check hosts and services and raise alerts. | Needs heavy set-up. Little user activity view. No built-in remote commands. | Simple agent set-up with a dashboard and quick actions. |

## 2.3 Base Paper and Conference Presentation

The base paper of this project is titled “Distributed Workstation Monitoring and Management System for College Laboratories” [8]. It describes four parts: workstation agents, safe communication, central processing and an admin dashboard. It stresses regular data collection, reports and safe admin control. The paper also admits that its judgement comes from design study and not from a large experiment.

The team presented the work at the AICTE sponsored International Conference ICAI-CEME 2026, held on 21 and 22 October 2026 at SIMATS Engineering, Thandalam, Chennai, under the title “Smart Focus – Admin Controlled Computer Monitoring and Management System”. The certificates are given at the end of this report.

## 2.4 Research Gap and Project Position

Earlier tools either watch only numbers or need heavy set-up. This project focuses on a simple working flow for college labs. A registered computer sends chosen health and activity data. A central service stores it and decides who may see it. The administrator reviews and controls the machines from a desktop application.

A design on paper is not enough. This project has real application code, database scripts and Edge Functions. Still, having code does not by itself prove that the system is ready for daily use or that it is fast. Those points need proper tests, which are described in Chapter 5.

## 2.5 Summary

The study shows that central visibility and safe control of lab computers are needed. This project meets that need with Electron applications and Supabase services. Access is checked in the screens and again in the database.

# CHAPTER 3. SYSTEM DESIGN

## 3.1 Existing System

In most labs today, the staff walk around and look at each computer. Different tools may be used for performance, software lists and reports. There is no single view of all computers and no single record of the actions taken on them. This causes delay, repeated effort and missed problems.

## 3.2 Proposed System

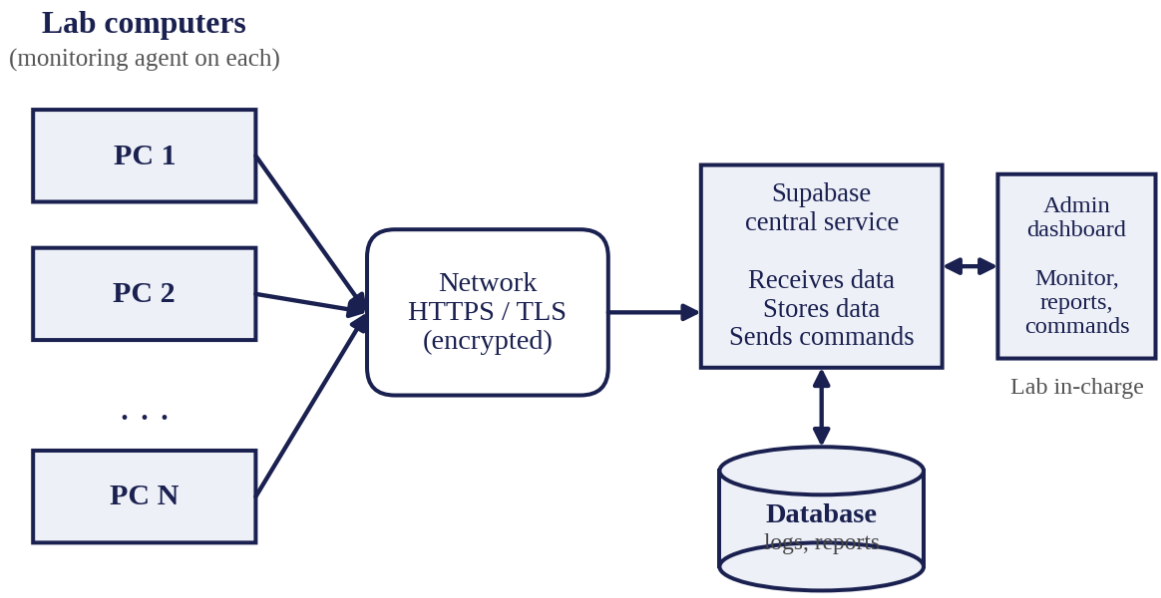
The proposed system has these parts:

- an **Admin desktop application** for the administrator;
- a **Viewer desktop application** that also works as the monitoring agent on a lab computer;
- **Supabase Auth and PostgreSQL** for login and data storage;
- **Realtime** updates so the dashboard changes when data changes;
- a **private storage bucket** for screenshots; and
- **Edge Functions** for jobs that need server-side power.

The administrator adds and manages the computers and views the data. The viewer application registers the local computer and runs the monitoring loop in the background. Supabase sits in the middle, checks every request and keeps the data safe.

## 3.3 System Architecture

Figure 1 shows the overall design. Many lab computers, each with a monitoring agent, send data over an encrypted network to the central service. The database keeps logs and reports. The admin dashboard shows the data and sends commands back.




*Agents send PC data to the central service. The admin views it and sends commands back.*

**Figure 1: High-level system architecture**

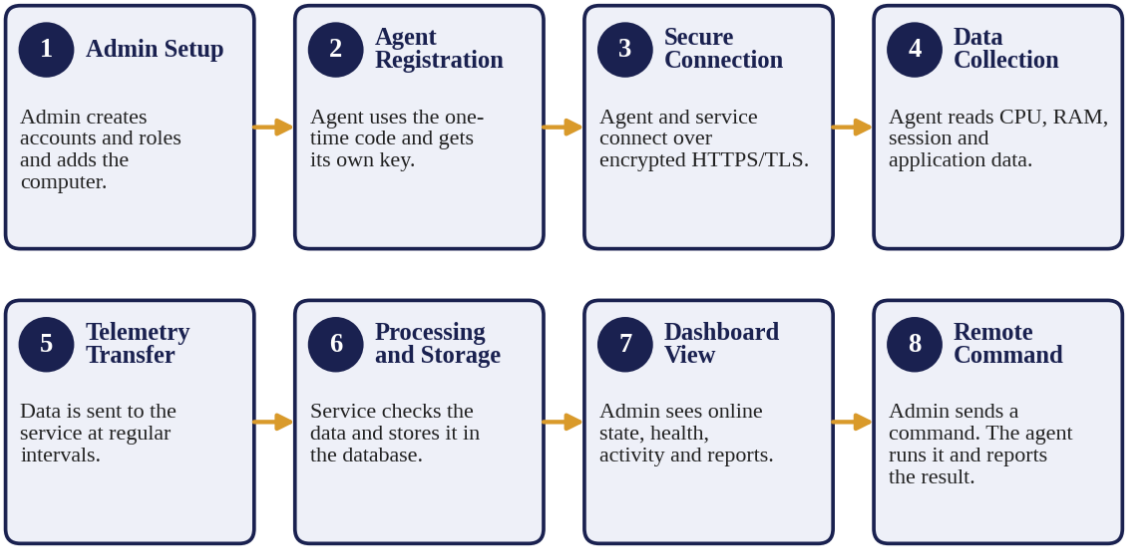
Table 2 lists the layers of the system and what each layer does.

**Table 2: Main software components**

| Layer | Component | Work done |
|---|---|---|
| Administrator | apps/admin-desktop | Dashboard, computer and lab management, activity, applications, sessions, reports, audit logs, users, settings and remote commands. |
| Endpoint | apps/viewer-desktop | Login, registration of the computer, and background monitoring. |
| Agent services | viewer-desktop/electron/agent | System metrics, active window and application activity, screenshots, command handling, offline queue, API client and local settings. |
| Backend | Supabase Auth, PostgreSQL, Realtime | Login, data storage, row-level security, live change messages and audit records. |
| Privileged operations | supabase/functions | Agent key issue and check, heartbeat and activity intake, screenshot upload, command results and viewer account set-up. |
| Extra application | apps/pc-agent | A stand-alone agent-only application. It is kept in the project but is paused in favour of the merged viewer-agent approach. |

## 3.4 Architecture Flow

Figure 2 shows the eight steps of the process, from admin set-up to a remote command. Steps 4 to 7 repeat all the time while the computers are on. Step 8 happens only when the administrator sends a command.




*Steps 4 to 7 repeat while the PCs are on. Step 8 happens whenever the admin sends a command.*

**Figure 2: Detailed process flow of the system**

In short, the flow in the code is as follows:

1. The administrator creates accounts and adds a computer in the Admin application. A one-time registration code is made.
2. The Viewer application uses this code to register the computer and receives the computer's own key.
3. The agent sends a heartbeat with CPU and memory use, and sends activity events, through the Edge Functions.
4. The backend checks each request, saves the records and applies the database access rules.
5. The Admin application listens for database changes and refreshes its views.
6. The administrator can send a supported command. The agent runs it and reports the result, which is then logged.

## 3.5 System Modules

The system is divided into six modules. Figure 3 shows them, and Table 3 gives their tasks.

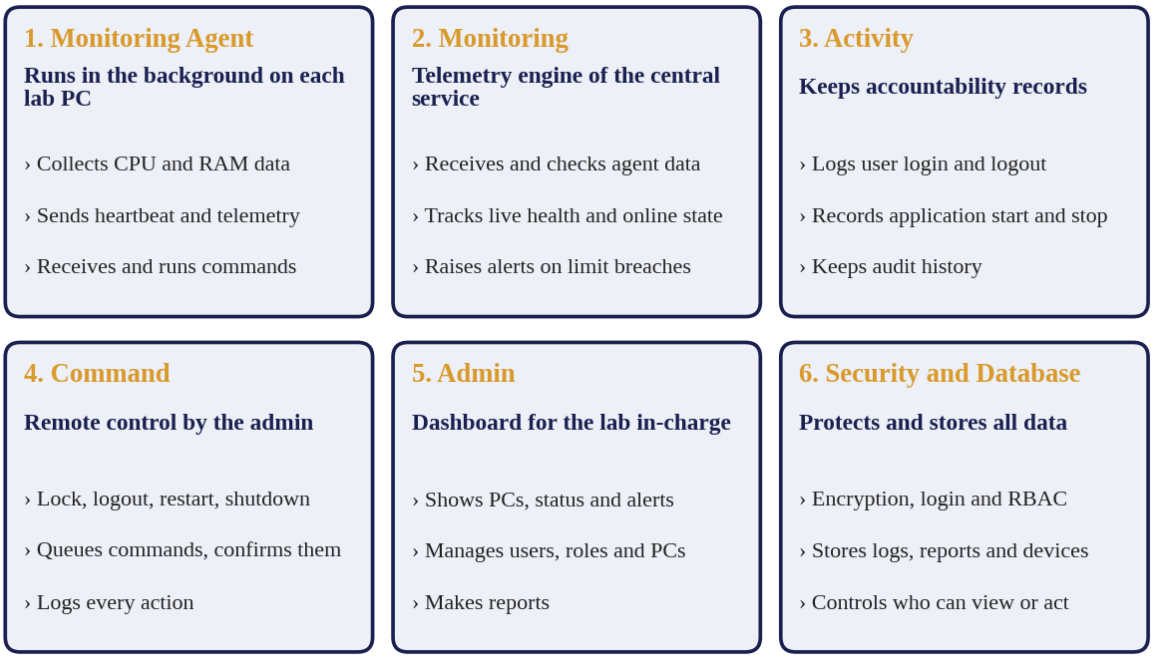



**Figure 3: Modules of the system and their functions**

**Table 3: Functional modules and their responsibilities**

| Module | Responsibility |
|---|---|
| **Monitoring Agent** | Runs in the background on every lab PC. Collects CPU and memory data, sends the heartbeat and receives and runs commands. |
| **Monitoring** | Receives and checks agent data, tracks the live health and the online or offline state of each PC. |
| **Activity** | Records user sessions, application use and audit history for each workstation. |
| **Command** | Lets the administrator lock, log out, restart or shut down a PC. Keeps each command and its result. |
| **Admin** | Dashboard for the lab in-charge. Manages users, roles and the PC list, and shows reports. |
| **Security and Database** | Handles login, role checks and row-level security. Stores logs, reports, user and device data. |

## 3.6 User Roles

The system has exactly two roles. Both are enforced by PostgreSQL row-level security [12] and not only by the screens.

- **ADMIN** – the only role that reads monitoring data: computers, labs, activity, applications, sessions, reports, screenshots and audit logs. An admin also manages computers, remote commands, viewer accounts and settings.

- **VIEWER** – has no dashboard. A viewer account is used to sign in on a lab PC and register that machine as an agent. The database rules still limit what a viewer could read to the labs an admin has assigned, as an extra layer of safety.

## 3.7 Functional Requirements

- Sign in users and tell apart the ADMIN and VIEWER roles.
- Register and manage computers and lab groups.
- Collect and show the status and chosen resource data of each workstation.
- Record activity, sessions, application events and audit history.
- Let an authorised administrator request the supported remote commands.
- Provide reports and CSV export for the data shown in the application.
- Apply row-level security and keep privileged work inside backend functions.

## 3.8 Non-Functional Requirements

- **Security:** secret keys must never be placed inside the desktop applications, and data access must be controlled by policies.
- **Reliability:** the agent should keep working through a short network break by queueing events and trying again.
- **Maintainability:** screens, services, agent modules and backend functions are kept in separate files.
- **Usability:** the administrator should see status and activity from one place.
- **Scalability:** database queries, live subscriptions, data frequency and storage use must be tested for the expected lab size.

## 3.9 Hardware and Software Requirements

**Table 4: Hardware and software requirements**

| Category | Requirement or technology |
|---|---|
| **Lab PC (endpoint)** | A supported desktop operating system, with the permissions needed for monitoring and commands. |
| **Admin computer** | A desktop that can run the Electron Admin application. |
| **Runtime** | Node.js 20 or newer and pnpm 10 or newer. |
| **Desktop framework** | Electron, React, TypeScript and Vite. |

| Category | Requirement or technology |
| --- | --- |
| **Data platform** | A Supabase project with PostgreSQL, Auth, Realtime, Storage and deployed Edge Functions. |
| **Security** | Supabase login, PostgreSQL row-level security, and local token storage with Electron safeStorage where available. |
| **Tools** | Supabase CLI for database scripts and functions; project scripts for build, type check, lint and tests. |

# CHAPTER 4. IMPLEMENTATION

## 4.1 Development Approach

The code is kept in one repository [9] that uses pnpm workspaces and the Turborepo task runner. It holds the two applications, shared packages, SQL scripts, Edge Functions, start-up data and release automation. Build, type check, lint and test commands are defined in the package files. The team did not keep execution logs of these commands, so this report does not claim that every build or test passed.

The work was planned from July to October 2026. Table 5 shows the plan.

**Table 5: Project schedule (July to October 2026)**

| Task | Planned dates |
|---|---|
| Requirement study and literature survey | 1 Jul – 15 Jul |
| Base paper study and problem definition | 10 Jul – 25 Jul |
| System design and architecture | 20 Jul – 10 Aug |
| Monitoring agent development | 5 Aug – 31 Aug |
| Central service and database | 20 Aug – 15 Sep |
| Admin dashboard and remote commands | 10 Sep – 30 Sep |
| Security: TLS, login and RBAC | 15 Sep – 10 Oct |
| Testing and lab deployment | 1 Oct – 20 Oct |
| Documentation and final review | 15 Oct – 31 Oct |

## 4.2 Administrator Desktop Application

The Admin application is in the folder apps/admin-desktop. Its React screens are: Dashboard, Computers, Computer Detail, Labs, Activity, Applications, Audit Logs, Sessions, Reports, Users and Settings, along with the Login page. Small service files handle the data work for each screen. Hooks run the queries and the live sync. Separate stores keep the login state and the connection state.

When an admin signs in, the application asks Supabase for the data the admin may read. It also subscribes to changes in the computers, sessions, activity events, application activity and remote commands tables, so the screens update without a manual refresh.

## 4.3 Viewer Desktop Application and Monitoring Agent

The Viewer application is in the folder apps/viewer-desktop. It has two jobs. When it is installed on a lab PC, its one page lets the computer register itself using a one-time code from the admin. After that, the main process of the application runs the monitoring loop in the background, whether or not anyone is signed in to the app.

The agent code is split into small files: API client, active-window reader, application monitor, command handler, local config store, offline queue, screenshot capture, system monitor and a controller that ties them together. The loop works as follows:

- **Heartbeat:** about every 15 seconds the agent sends the CPU percentage and memory percentage.
- **Application activity:** the agent reads the active window and records when an application opens, switches or closes. On Windows it uses a small PowerShell call to the Win32 foreground-window function. On other systems it uses the active-win library.
- **Screenshots:** one compressed still picture is taken at a fixed gap. The default is 60 seconds and the lowest allowed gap is 30 seconds. There is no video or live view.
- **Commands:** the agent runs only four fixed actions (lock, logout, restart, shutdown) and reports the result.
- **Offline queue:** if the network fails, events are saved to disk and sent later. The reconnect delay starts at 1 second and doubles up to 60 seconds.
- **Safe token:** the agent key is saved with Electron safeStorage, which uses the operating system key store, and is never kept as plain text.

If an admin deletes a computer or cancels its key, the agent shows this and offers “Reset registration”. This clears the old local key so the computer can register again with a new code.

## 4.4 Standalone PC Agent

The folder apps/pc-agent holds a separate Electron application with the same monitoring code. It is a plain agent without the viewer screens. The project README says this application is paused, because the merged viewer-agent application is now used. It stays in the repository for reference and should not be treated as the main way to deploy the system.

## 4.5 Supabase Functions and Database

The supabase folder holds the SQL scripts and the Edge Functions. The SQL scripts create the tables, helper functions, triggers, row-level security policies, realtime settings, metric columns, report functions, viewer lab assignments, screenshot storage and lab-scoped viewer access.

Edge Functions run on Deno and use the service-role key, which stays inside Supabase and never reaches a desktop application. Table 6 lists them.

**Table 6: Edge Functions of the system**

| Function | Purpose |
|---|---|
| **agent-register** | Uses a one-time registration code and issues a long-lived agent key. |
| **agent-heartbeat** | Updates the online status and the CPU and memory values. Returns pending commands. |
| **agent-activity** | Accepts batches of application and session events. |
| **agent-screenshot** | Stores a screenshot in the private bucket. |
| **agent-command-result** | Receives the result of a remote command (succeeded or failed). |
| **admin-create-viewer** | Creates a viewer account. |
| **admin-update-viewer** | Edits a viewer account. |
| **admin-delete-viewer** | Deletes a viewer account. |
| **offline-sweep** | Marks a computer OFFLINE when its heartbeat has expired (45 seconds by default). Runs on a schedule and is protected by a secret. |

The database holds the tables listed in Table 7.

**Table 7: Main database tables**

| Table | What it stores |
|---|---|
| **profiles** | One row for each admin or viewer user, with the role. |
| **labs** | Lab names and groups. |
| **computers** | Each computer: code, name, lab, host name, IP address, operating system, status, last seen time, CPU and memory values. |
| **computer_registration_codes** | Hashed one-time codes, expiry time and use time. |
| **computer_credentials** | Hashed agent keys and the time a key was cancelled. |
| **sessions** | Login sessions on a monitored computer. |
| **applications, application_activity** | Applications seen and the time each was used. |
| **activity_events** | Events such as APP_OPEN, APP_CLOSE, APP_SWITCH, SESSION_START, SESSION_END, PC_ONLINE and PC_OFFLINE. |
| **remote_commands** | Commands with status PENDING, SENT, ACKNOWLEDGED, SUCCEEDED, FAILED or TIMED_OUT. |
| **audit_logs** | An append-only record of admin actions and command results. |
| **system_settings, viewer_lab_assignments** | Monitoring settings and the labs assigned to each viewer. |

## 4.6 Security Design

- **Two roles only:** ADMIN and VIEWER, checked by PostgreSQL row-level security as well as by the screens.
- **Keys are hashed:** registration codes and agent keys are stored only as hashes. The real key is shown to the agent one time.
- **One-time codes:** a registration code can be used once and expires.
- **No service key in clients:** the service-role key exists only inside Edge Functions.
- **Fixed commands:** no free-text command can be run. Only four fixed actions are allowed.
- **Private screenshots:** screenshots go to a private bucket and an admin views them through a short-lived signed link.
- **Audit trail:** every admin action and command result is written to the audit_logs table.

## 4.7 Main Modules

**Table 8: Main modules and their code locations**

| Module | Location in the project | Purpose |
|---|---|---|
| Dashboard and inventory | admin-desktop/src/pages/Dashboard.tsx, Computers.tsx, Labs.tsx | Summary of workstation status and management of computers and labs. |
| Computer details | admin-desktop/src/pages/ComputerDetail.tsx | Information and actions for one computer. |
| Activity and sessions | pages/Activity.tsx, Applications.tsx, Sessions.tsx | Recorded activity, application events and sessions. |
| Reports and audit | pages/Reports.tsx, AuditLogs.tsx | Reports and admin audit view. |
| Command service | admin-desktop/src/services/commands.ts and agent/command-handler.ts | Sends command requests and runs them on the PC. |
| System monitoring | viewer-desktop/electron/agent/system-monitor.ts | Reads CPU and memory use. |
| Activity capture | agent/active-window-windows.ts, application-monitor.ts | Watches active window and application changes. |
| Offline reliability | agent/offline-queue.ts | Queues events during a network break and sends them later. |
| Secure data access | supabase/migrations and supabase/functions | Database rules and server-side work. |

## 4.8 Data and Event Flow

Data moves from the lab PC to Supabase, and the admin application reads it from there. A remote command moves the other way. Figure 4 shows this for one command.

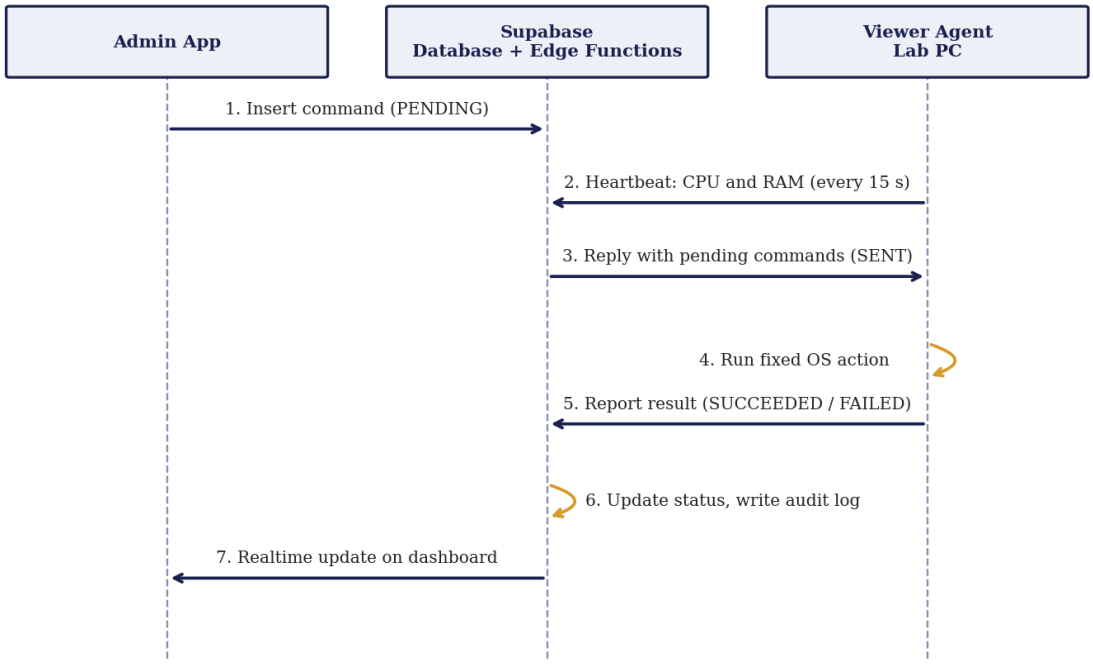



**Figure 4: Remote command workflow between admin, Supabase and the agent**

The admin application only creates a request with the status PENDING. The agent finds it at its next heartbeat. Only the Edge Function can change the status to SUCCEEDED or FAILED. How new the dashboard looks depends on the network, the heartbeat gap, the realtime connection and the backend set-up.

## 4.9 Repository Structure

```
Mini-Project/
  apps/
    admin-desktop/     Electron app for ADMIN
    viewer-desktop/    Electron app for VIEWER + agent
    pc-agent/          stand-alone agent (paused)
  packages/
    types/             shared TypeScript types
    validation/        shared Zod schemas
    config/            shared constants
    ui/                small shared components
  supabase/
    migrations/        SQL schema, policies, reports
    functions/         Edge Functions (Deno)
    seed.sql           creates the first ADMIN
  scripts/             database helper scripts
  .github/workflows/   release automation
```

**Figure 5: Repository application structure**

## 4.10 Build and Release

Each application is built with pnpm and packed with electron-builder. A GitHub Actions workflow builds both applications for Windows (.exe) and macOS (.dmg) and publishes them when a version tag is pushed. The set-up steps are: install the packages, copy the example environment file and add the Supabase URL and public key, run the database migration script, seed the first admin, and deploy the Edge Functions.

# CHAPTER 5. TESTING, RESULTS AND DISCUSSION

## 5.1 Testing Strategy

Testing must cover two things: how the applications behave, and whether the backend allows only the right users to do the right things. The project defines build, type check, lint and test commands. However, no test report, command output, deployment log or speed measurement was kept. For this reason, this chapter gives a test plan and does not show pass or fail results as if they were finished experiments. Table 9 gives the plan.

**Table 9: Test plan for the system**

| Test area | Test case | Evidence to record |
|---|---|---|
| **Login** | Sign in as ADMIN and as VIEWER and check the role limits. | Test account, expected access, actual result and time. |
| **Registration** | Register a PC with a valid one-time code. Reject a wrong or expired code. | Registration log and the new device record. |
| **Heartbeat** | Check that CPU and memory values are saved and that the status changes after the timeout. | Agent log and database time stamps. |
| **Activity** | Open, switch and close applications and check the events. | Application event records and the screen view. |
| **Screenshots** | Check the timed capture, private storage and access limits. | Storage record and allowed and denied access checks. |
| **Remote commands** | Test lock, logout, restart and shutdown in a safe test set-up. | Command status, agent result and audit record. |
| **Offline recovery** | Cut the network for a short time, reconnect, and check that queued events are sent. | Queue log and the resulting records. |
| **Row-level security** | Try to cross role and lab limits with test accounts. | Allow and deny table with SQL or API logs. |
| **Performance** | Measure CPU, memory, network use, event delay and the number of PCs at once. | Repeatable test set-up and its results. |

## 5.2 Results

### 5.2.1 Implemented Output

The project code contains the Admin application, the Viewer and agent workflow, the database scripts and the Edge Functions. The README and the project presentation describe the heartbeat, activity, screenshot, command, login and access-control behaviour. Figure 6 shows the sign-in screen of the Admin application, which is the entry point of the system.

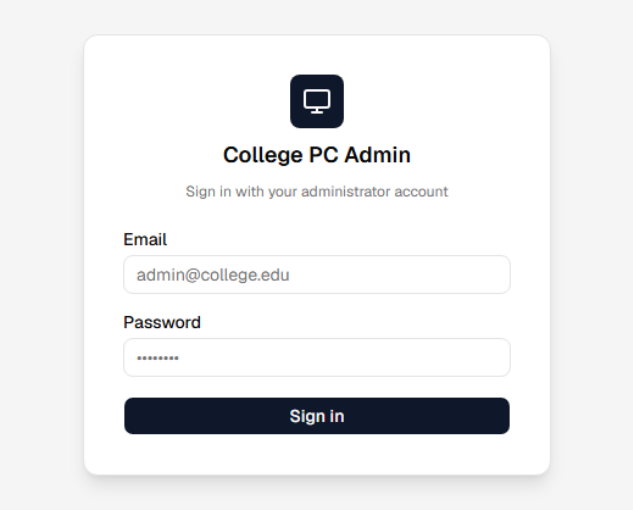



**Figure 6: Sign-in screen of the College PC Admin application**

### 5.2.2 Default Monitoring Settings

The shared configuration package fixes the default values that control how the agent behaves. Table 10 lists them.

**Table 10: Default monitoring settings from the configuration package**

| Setting | Default value |
|---|---|
| **Heartbeat gap** | 15 seconds |
| **Offline limit (no heartbeat)** | 45 seconds |
| **Screenshot gap** | 60 seconds (lowest allowed: 30 seconds) |
| **Offline queue size** | 500 events (oldest dropped first) |
| **Reconnect delay** | Starts at 1 second, doubles, up to 60 seconds |
| **Activity retention** | 90 days |
| **Audit log retention** | 365 days |
| **Allowed remote commands** | LOCK, LOGOUT, RESTART, SHUTDOWN |

### 5.2.3 What the Evidence Shows

The code proves that all main parts exist: an admin interface, a viewer and agent workflow, database scripts with security rules, and Edge Functions. It shows how each part is meant to work. It does not prove exact speed, accuracy or reliability. No such numbers are claimed here. They should be measured in a later test, as listed in Table 9.

## 5.3 Main Source Code

This section gives the main source code of the project. It starts with the agent on the lab PC, then the server function, then the admin service and the database. Long comments in the code are wrapped to fit the page.

### 5.3.1 Shared Configuration (packages/config/src/index.ts)

These constants set the default times and limits used by the agent and the backend.

```
export const DEFAULT_HEARTBEAT_INTERVAL_SECONDS = 15;
export const DEFAULT_OFFLINE_THRESHOLD_SECONDS = 45;
export const DEFAULT_ACTIVITY_RETENTION_DAYS = 90;
export const DEFAULT_AUDIT_RETENTION_DAYS = 365;

// Periodic, not live: a fixed floor keeps this from ever becoming a
// continuous screen-recording feature.
export const DEFAULT_SCREENSHOT_INTERVAL_SECONDS = 60;
export const MIN_SCREENSHOT_INTERVAL_SECONDS = 30;

export const AGENT_OFFLINE_QUEUE_MAX_EVENTS = 500;
export const AGENT_RECONNECT_BASE_DELAY_MS = 1000;
export const AGENT_RECONNECT_MAX_DELAY_MS = 60_000;
```

### 5.3.2 System Monitor (agent/system-monitor.ts)

This file reads the CPU and memory use. CPU use needs two samples taken a short time apart, because the operating system gives only running totals.

```
import os from "node:os";

interface CpuSample {
  idle: number;
  total: number;
}

function sampleCpus(): CpuSample {
  let idle = 0;
  let total = 0;
  for (const cpu of os.cpus()) {
    for (const type of Object.keys(cpu.times) as (keyof typeof cpu.times)[]) {
      total += cpu.times[type];
    }
    idle += cpu.times.idle;
  }
  return { idle, total };
}

// CPU usage requires two samples over an interval (instantaneous
// snapshots of os.cpus() are cumulative counters, not a point-in-time
// percentage).
export function getCpuUsagePercent(sampleWindowMs = 300): Promise<number> {
  const start = sampleCpus();
  return new Promise((resolve) => {
    setTimeout(() => {
      const end = sampleCpus();
      const idleDelta = end.idle - start.idle;
      const totalDelta = end.total - start.total;
      const usage = totalDelta === 0 ? 0 : 100 * (1 - idleDelta / totalDelta);
```

```
      resolve(Math.max(0, Math.min(100, Math.round(usage * 10) / 10)));
    }, sampleWindowMs);
  });
}

export function getMemoryUsagePercent(): number {
  const total = os.totalmem();
  const free = os.freemem();
  return Math.round(((total - free) / total) * 1000) / 10;
}

export function getHostname(): string {
  return os.hostname();
}

export function getOperatingSystem(): string {
  return `${os.type()} ${os.release()} (${os.arch()})`;
}

export function getCurrentUsername(): string {
  try {
    return os.userInfo().username;
  } catch {
    return "unknown";
  }
}
```

### 5.3.3 Agent Controller: Heartbeat, Commands and Activity (agent/controller.ts)

The controller sends the heartbeat, runs any commands that come back, and sends batches of activity. If the heartbeat fails, it waits and tries again with a longer delay each time.

```
  private async heartbeat(): Promise<void> {
    if (!this.config) return;

    try {
      const [cpuUsage, memoryUsage] = await Promise.all([
        getCpuUsagePercent(),
        Promise.resolve(getMemoryUsagePercent()),
      ]);

      const result = await sendHeartbeat(this.config.agentToken, {
        cpuUsage,
        memoryUsage,
        agentVersion: AGENT_VERSION,
      });

      this.reconnectDelayMs = AGENT_RECONNECT_BASE_DELAY_MS;
      this.setState({
        status: "online",
        computerCode: this.config.computerCode,
        computerId: this.config.computerId,
        lastHeartbeatAt: new Date().toISOString(),
        lastError: null,
      });

      for (const command of result.commands) {
        void this.handleCommand(command.commandId, command.commandType);
      }

      this.scheduleHeartbeat(result.heartbeatIntervalSeconds * 1000);
    } catch (error) {
      const message = error instanceof Error ? error.message : "Heartbeat failed.";
      this.setState({ status: "offline", lastError: message });
```

```
      this.scheduleHeartbeat(this.reconnectDelayMs);
      this.reconnectDelayMs = Math.min(this.reconnectDelayMs * 2,
          AGENT_RECONNECT_MAX_DELAY_MS);
    }
  }

  private async handleCommand(
    commandId: string,
    commandType: Parameters<typeof executeCommand>[0],
  ): Promise<void> {
    if (!this.config) return;
    const outcome = await executeCommand(commandType);
    try {
      await sendCommandResult(this.config.agentToken, {
        commandId,
        status: outcome.status,
        result: outcome.result,
      });
    } catch (error) {
      console.warn("[agent] Failed to report command result:", error);
    }
  }

  private async flushActivity(): Promise<void> {
    if (!this.config) return;
    const queued = this.offlineQueue.drain();
    const batch = [...queued, ...this.pendingEvents];
    if (batch.length === 0) return;
    this.pendingEvents = [];

    try {
      await sendActivityBatch(this.config.agentToken, batch);
    } catch (error) {
      console.warn("[agent] Failed to send activity batch, queuing offline:", error);
      this.offlineQueue.requeue(batch);
    }
  }

  private setState(patch: Partial<AgentState>): void {
    this.state = { ...this.state, ...patch };
    this.onStateChange(this.state);
  }
}
```

### 5.3.4 Command Handler (agent/command-handler.ts)

Only four fixed actions can be run. Each one is a single fixed command for the operating system. No other command can be sent.

```
import { exec } from "node:child_process";
import os from "node:os";
import type { RemoteCommandType } from "@college-monitor/types";

export interface CommandExecutionResult {
  status: "SUCCEEDED" | "FAILED";
  result: Record<string, unknown>;
}

// Only these four fixed, non-parameterized OS actions are ever executed —
// there is no arbitrary shell/command execution path anywhere in the
// agent. Each maps to a single hard-coded platform command.
const WINDOWS_COMMANDS: Record<RemoteCommandType, string> = {
  LOCK: "rundll32.exe user32.dll,LockWorkStation",
  LOGOUT: "shutdown /l",
  RESTART: "shutdown /r /t 5",
```

```
  SHUTDOWN: "shutdown /s /t 5",
};

const MACOS_COMMANDS: Record<RemoteCommandType, string> = {
  LOCK: "/System/Library/CoreServices/Menu\\
      Extras/User.menu/Contents/Resources/CGSession -suspend",
  LOGOUT: "osascript -e 'tell application \"System Events\" to log out'",
  RESTART: "osascript -e 'tell application \"System Events\" to restart'",
  SHUTDOWN: "osascript -e 'tell application \"System Events\" to shut down'",
};

const LINUX_COMMANDS: Record<RemoteCommandType, string> = {
  LOCK: "loginctl lock-session",
  LOGOUT: "loginctl terminate-session $XDG_SESSION_ID",
  RESTART: "systemctl reboot",
  SHUTDOWN: "systemctl poweroff",
};

function commandsForPlatform(): Record<RemoteCommandType, string> {
  switch (os.platform()) {
    case "win32":
      return WINDOWS_COMMANDS;
    case "darwin":
      return MACOS_COMMANDS;
    default:
      return LINUX_COMMANDS;
  }
}

export function executeCommand(type: RemoteCommandType): Promise<CommandExecutionResult>
     {
  const command = commandsForPlatform()[type];

  return new Promise((resolve) => {
    exec(command, { timeout: 15_000 }, (error, stdout, stderr) => {
      if (error) {
        resolve({
          status: "FAILED",
          result: { error: error.message, stderr: stderr?.slice(0, 500) },
        });
        return;
      }
      resolve({ status: "SUCCEEDED", result: { stdout: stdout?.slice(0, 500) } });
    });
  });
}
```

### 5.3.5 Offline Queue (agent/offline-queue.ts)

When the backend cannot be reached, events are kept in a small file on disk. The queue has a size limit, so a long network failure cannot fill the disk.

```
import { app } from "electron";
import fs from "node:fs";
import path from "node:path";
import { AGENT_OFFLINE_QUEUE_MAX_EVENTS } from "@college-monitor/config";
import type { ActivityEventPayload } from "./api-client";

// A small, capped, disk-backed queue of activity events collected while
// the backend is unreachable. Bounded so a prolonged outage can't grow
// this without limit — oldest events are dropped first once full, which
// is an acceptable tradeoff for a monitoring feed (recent activity is
// far more actionable than an unbounded backlog).
function queuePath(): string {
  return path.join(app.getPath("userData"), "offline-queue.json");
```

```
}

export class OfflineQueue {
  private events: ActivityEventPayload[] = [];

  constructor() {
    this.load();
  }

  push(event: ActivityEventPayload): void {
    this.events.push(event);
    if (this.events.length > AGENT_OFFLINE_QUEUE_MAX_EVENTS) {
      this.events.splice(0, this.events.length - AGENT_OFFLINE_QUEUE_MAX_EVENTS);
    }
    this.persist();
  }

  drain(): ActivityEventPayload[] {
    const drained = this.events;
    this.events = [];
    this.persist();
    return drained;
  }

  requeue(events: ActivityEventPayload[]): void {
    this.events = [...events, ...this.events].slice(-AGENT_OFFLINE_QUEUE_MAX_EVENTS);
    this.persist();
  }

  get size(): number {
    return this.events.length;
  }

  private load(): void {
    try {
      this.events = JSON.parse(fs.readFileSync(queuePath(), "utf8")) as
          ActivityEventPayload[];
    } catch {
      this.events = [];
    }
  }

  private persist(): void {
    try {
      fs.mkdirSync(path.dirname(queuePath()), { recursive: true });
      fs.writeFileSync(queuePath(), JSON.stringify(this.events));
    } catch (error) {
      console.warn("[agent] Failed to persist offline queue:", error);
    }
  }
}
```

### 5.3.6 Screenshot Capture (agent/screenshot.ts)

This file takes one compressed still picture of the main screen each time it is called. It has no video or live-view code.

```
import { desktopCapturer } from "electron";

export type ScreenshotResult =
  | { ok: true; imageBase64: string }
  | { ok: false; reason: string };
```

```
// Captures a single compressed still frame of the primary display. This
// is intentionally a one-shot snapshot, called on a timer by the agent
// controller (see MIN_SCREENSHOT_INTERVAL_SECONDS) — there is no
// continuous capture, video, or recording path anywhere in this module.
//
// macOS requires the user to grant this app "Screen Recording" access
// (System Settings -> Privacy & Security -> Screen Recording) — without
// it, getSources() returns no usable thumbnail. Returns a reason string
// on any failure instead of silently swallowing it, since a packaged app
// has no visible console to catch a console.warn.
export async function captureScreenshotBase64(): Promise<ScreenshotResult> {
  try {
    const sources = await desktopCapturer.getSources({
      types: ["screen"],
      thumbnailSize: { width: 1280, height: 800 },
    });

    if (sources.length === 0) {
      return { ok: false, reason: "No screen sources returned by the OS." };
    }

    const primary = sources[0];
    if (!primary || primary.thumbnail.isEmpty()) {
      return { ok: false, reason: "Screen source thumbnail was empty (permission not
          granted?)." };
    }

    const jpeg = primary.thumbnail.toJPEG(60);
    return { ok: true, imageBase64: jpeg.toString("base64") };
  } catch (error) {
    return { ok: false, reason: error instanceof Error ? error.message : String(error)
        };
  }
}
```

### 5.3.7 Heartbeat Edge Function (supabase/functions/agent-heartbeat/index.ts)

This server function checks the agent, saves the CPU and memory values, marks the computer ONLINE and returns any pending commands.

```
// Periodic PC Agent heartbeat. Updates online status + live metrics, and
// hands back any PENDING remote commands for this computer so the agent
// can execute them (marking them SENT here — the agent reports the final
// result via agent-command-result).
import { z } from "npm:zod@3.24.1";
import { createAdminClient } from "../_shared/supabase-admin.ts";
import { requireAgent } from "../_shared/agent-auth.ts";
import { jsonError, jsonSuccess, handleUnknownError } from "../_shared/response.ts";
import { handleCorsPreflight } from "../_shared/cors.ts";

const bodySchema = z.object({
  cpuUsage: z.number().min(0).max(100),
  memoryUsage: z.number().min(0).max(100),
  ipAddress: z.string().max(64).optional(),
  agentVersion: z.string().max(32).optional(),
});

Deno.serve(async (req) => {
  const preflight = handleCorsPreflight(req);
  if (preflight) return preflight;
  if (req.method !== "POST") return jsonError("NOT_FOUND", "Not found.", 404);

  try {
    const admin = createAdminClient();
    const agent = await requireAgent(req, admin);
```

```
    const parsed = bodySchema.safeParse(await req.json());
    if (!parsed.success) {
      return jsonError("VALIDATION_ERROR", "Invalid request body.", 422,
          parsed.error.flatten());
    }
    const { cpuUsage, memoryUsage, ipAddress, agentVersion } = parsed.data;

    const { data: existing } = await admin
      .from("computers")
      .select("status")
      .eq("id", agent.computerId)
      .maybeSingle();

    const wasOffline = !existing || existing.status !== "ONLINE";
    const now = new Date().toISOString();

    const update: Record<string, unknown> = {
      status: "ONLINE",
      last_seen_at: now,
      cpu_usage: cpuUsage,
      memory_usage: memoryUsage,
    };
    if (ipAddress) update.ip_address = ipAddress;
    if (agentVersion) update.agent_version = agentVersion;

    const { error: updateError } = await admin
      .from("computers")
      .update(update)
      .eq("id", agent.computerId);
    if (updateError) throw updateError;

    if (wasOffline) {
      await admin.from("activity_events").insert({
        computer_id: agent.computerId,
        event_type: "PC_ONLINE",
        occurred_at: now,
      });
    }

    const { data: pendingCommands } = await admin
      .from("remote_commands")
      .select("id, command_type")
      .eq("computer_id", agent.computerId)
      .eq("status", "PENDING");

    if (pendingCommands && pendingCommands.length > 0) {
      await admin
        .from("remote_commands")
        .update({ status: "SENT" })
        .in(
          "id",
          pendingCommands.map((c) => c.id),
        );
    }

    return jsonSuccess({
      heartbeatIntervalSeconds: 15,
      commands: (pendingCommands ?? []).map((c) => ({ commandId: c.id, commandType:
          c.command_type })),
    });
  } catch (error) {
    return handleUnknownError(error);
  }
});
```

### 5.3.8 Admin Command Service (admin-desktop/src/services/commands.ts)

The admin application only creates a PENDING request and writes an audit log. The agent picks the request up at its next heartbeat.

```
import type { RemoteCommand, RemoteCommandType } from "@college-monitor/types";
import { supabase } from "../lib/supabase";
import { mapRemoteCommand } from "../lib/mappers";
import { writeAuditLog } from "./audit";

// Only ever creates a PENDING request — the RLS insert policy on
// remote_commands enforces that server-side too. The PC Agent picks this
// up on its next heartbeat and reports the result via the
// agent-command-result Edge Function, which is the only path that can
// move status to SUCCEEDED/FAILED.
export async function requestCommand(
  computerId: string,
  commandType: RemoteCommandType,
  adminId: string,
): Promise<RemoteCommand> {
  const { data, error } = await supabase
    .from("remote_commands")
    .insert({ computer_id: computerId, requested_by: adminId, command_type: commandType,
         status: "PENDING" })
    .select()
    .single();
  if (error) throw error;

  await writeAuditLog({
    adminId,
    action: `COMMAND_${commandType}_REQUESTED`,
    computerId,
    metadata: { commandId: data.id },
  });

  return mapRemoteCommand(data);
}
```

### 5.3.9 Database Table and Security Rules (supabase/migrations)

The first listing is the computers table. The second shows the helper function that checks for an admin and the first row-level security policies. Later scripts add lab-scoped access for viewers.

```
create table computers (
  id uuid primary key default gen_random_uuid(),
  computer_code text not null unique,
  name text not null,
  lab_id uuid references labs (id) on delete set null,
  hostname text,
  ip_address text,
  operating_system text,
  agent_version text,
  status computer_status not null default 'OFFLINE',
  last_seen_at timestamptz,
  is_enabled boolean not null default true,
  registered_at timestamptz,
  updated_at timestamptz not null default now()
);

create index idx_computers_status on computers (status);
create index idx_computers_last_seen_at on computers (last_seen_at);
```

```
create or replace function is_admin()
returns boolean
language sql
security definer
set search_path = public
stable
as $$
  select coalesce(auth_role() = 'ADMIN', false);
$$;



create policy profiles_admin_all on profiles
  for all using (is_admin()) with check (is_admin());

create policy profiles_self_select on profiles
  for select using (id = auth.uid());

-- labs ------------------------------------------------------------------

create policy labs_admin_all on labs
```

## 5.4 Limitations

- The system needs a correctly set up Supabase project with policies, storage and deployed Edge Functions.
- Monitoring and remote commands depend on the permissions and behaviour of the operating system on each PC.
- Screenshots and application records are private data. They need a clear college policy, access limits, retention rules and privacy care.
- No test shows how the system works with hundreds of computers.
- The folder apps/pc-agent is paused, so the deployment notes must say which application is the main one.

## 5.5 Discussion

The project is split into clear parts: screens, data collection, database rules and server-side work. This makes it easier to understand and to change. The biggest practical gain is that an administrator can read the information about all computers in one place instead of visiting each machine.

Before real use, the college should check the security set-up, test each remote command on the operating systems it supports, measure the speed with a realistic number of computers, and decide the rules for consent and for how long data is kept.

# CHAPTER 6. CONCLUSION AND FUTURE WORK

## 6.1 Conclusion

The College PC Monitoring and Management System gives a working base for watching lab computers from one place. It joins Electron desktop applications, React and TypeScript screens, a monitoring agent and Supabase services for login, storage, live updates and safe server work. The code covers the computer list, health data, activity, sessions, reports, audit records and remote commands.

The report describes the system as it is built and keeps built features separate from results that are still to be measured. With more testing and clear privacy rules, the system can grow into a dependable tool for college labs.

## 6.2 Future Enhancement

- Run repeatable speed and load tests with a fixed number of computers, and record CPU, memory, network use and event delay.
- Build an automatic test set for login, registration, commands, row-level security and offline recovery.
- Write better deployment notes for environment values, the order of database scripts, function secrets and PC set-up.
- Set privacy controls for screenshots and active-window data, including consent, retention, access logs and deletion.
- Add monitoring for agent health, old keys, backend outages and failed commands.
- Check accessibility, error messages and recovery on every supported operating system.
- Add alerts based on limits and, later, simple prediction of faults.

# REFERENCES


[1] E. Al-Shaer, H. Abdel-Wahab, and K. Maly, “HiFi: A new monitoring architecture for distributed systems management,” in Proc. 19th IEEE Int. Conf. Distributed Computing Systems (ICDCS), 1999, pp. 171–178.

[2] G. A. McGilvary, A. Barker, and M. Atkinson, “Ad hoc cloud computing,” in Proc. IEEE 8th Int. Conf. Cloud Computing (CLOUD), 2015, pp. 1063–1068.

[3] M. L. Massie, B. N. Chun, and D. E. Culler, “The Ganglia distributed monitoring system: Design, implementation, and experience,” Parallel Computing, vol. 30, no. 7, pp. 817–840, 2004.

[4] M. J. Sottile and R. G. Minnich, “Supermon: A high-speed cluster monitoring system,” in Proc. IEEE Int. Conf. Cluster Computing, 2002, pp. 39–46.

[5] S. Zanikolas and R. Sakellariou, “A taxonomy of grid monitoring systems,” Future Generation Computer Systems, vol. 21, no. 1, pp. 163–188, 2005.

[6] W. Barth, Nagios: System and Network Monitoring, 2nd ed. San Francisco, CA, USA: No Starch Press, 2008.

[7] A. S. Tanenbaum and M. Van Steen, Distributed Systems: Principles and Paradigms, 2nd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2007.

[8] M. N. A. Rahman, P. S. Jeevanandham, P. Hariharan, and D. Bairavi S, “Smart Focus – Admin controlled computer monitoring and management system,” presented at the AICTE Sponsored Int. Conf. on AI-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026), SIMATS Engineering, Chennai, India, Oct. 21–22, 2026.

[9] “College PC Monitoring & Management System,” project repository and README (Mini-Project), 2026.

[10] “Electron documentation,” OpenJS Foundation. [Online]. Available: https://www.electronjs.org/docs/latest

[11] “Supabase documentation,” Supabase Inc. [Online]. Available: https://supabase.com/docs

[12] “Row security policies,” PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

# CONFERENCE PAPER CERTIFICATES



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

**Noorul Aatif Rahman M**

vels univesity

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

*Smart focus- Admin Controlled computer monitoring and management system*

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

**Jeevanandham P S**

vels univesity

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

*Smart focus- Admin Controlled computer monitoring and management system*

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**

# CONFERENCE PAPER CERTIFICATES



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

## CERTIFICATE OF PARTICIPATION

This certificate is presented to

**Hariharan p**

vels univesity

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

*Smart focus- Admin Controlled computer monitoring and management system*

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

## CERTIFICATE OF PARTICIPATION

This certificate is presented to

**Mrs.Divya Bairavi S ME.,(PhD) Assistant Professor**

vels univesity

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

*Smart focus- Admin Controlled computer monitoring and management system*

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**